

Universidad Nacional Autónoma de México



Facultad de Ingeniería

Laboratorio de Arquitectura y Organización de Computadoras

Grupo: 07 - Semestre: 2027-1

Reporte Práctica 7.

Procesador CISC 68HC11

Fecha de Entrega

13 de Octubre del 2026

Profesor

Ing. Julio Cesar Cruz Estrada

Integrantes:

Espinoza Matamoros Percival Ulises

Montiel Juárez Oscar Iván

Veraza García Amy Valentina

1. Objetivo

Analizar y ampliar el funcionamiento de un procesador CISC de 8 bits basado en el 68HC11, mediante la preparación de su memoria de microprograma y la ejecución de programas en lenguaje máquina. Verificar en simulación las cargas de datos, la suma de acumuladores, la comparación, los saltos y las transferencias entre registros y memoria.

2. Introducción

Una instrucción del procesador requiere varias operaciones internas: colocar una dirección en el bus, leer un dato, seleccionar los operandos de la unidad de procesamiento aritmético (UPA), guardar el resultado y actualizar las banderas. En el proyecto utilizado, estas operaciones se coordinan mediante palabras de control almacenadas en la memoria del secuenciador.

La práctica se desarrolló a partir del proyecto base y de los materiales de apoyo del profesor. Se trabajó en dos niveles: la microprogramación define cómo se ejecuta cada instrucción y el código máquina determina qué instrucciones ejecuta el procesador. Se presentan tres ejercicios y seis pruebas: suma repetitiva y carga de `FF`, comparación de registros con valores iguales y diferentes, y comparación con acceso a memoria en ambos casos.

3. Desarrollo

3.1. Organización del procesador

La arquitectura dispone de dos acumuladores de 8 bits, A y B, una UPA, un registro de códigos de condición (CCR), un contador de programa (PC), registros auxiliares y una unidad de control microprogramada. Los datos ocupan bytes de 8 bits y las direcciones que manejan el PC y los saltos extendidos tienen 16 bits. La RAM implementada contiene 256 posiciones de 8 bits; los programas utilizan el intervalo `0000` a `00FF`.

El PC indica la dirección del programa en RAM. La señal `Edo_Pres` indica la dirección de la microinstrucción dentro del secuenciador. Son direcciones distintas: el PC avanza según los bytes de la instrucción, mientras que el secuenciador recorre los pasos necesarios para ejecutarla.

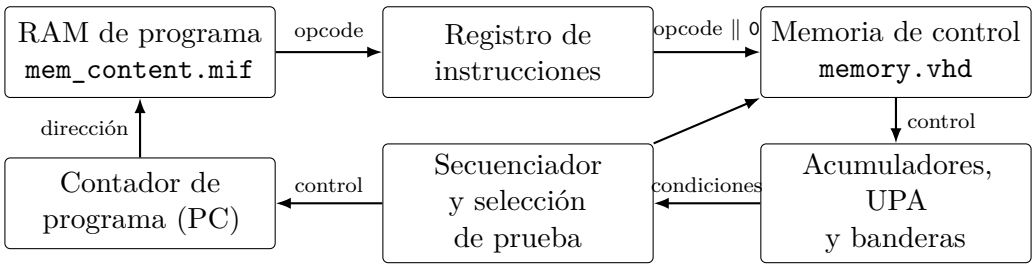


Figura 1: Esquema funcional simplificado de la relación entre programa, microprograma y ruta de datos. Se omiten buses y conexiones auxiliares para destacar el flujo de control.

3.2. Memoria de programa y memoria de control

El archivo `mem_content.mif` inicializa la RAM declarada en `RAM/ram.vhd`. Cada posición contiene un opcode, un operando o un dato. Su enlace se establece con el atributo siguiente:

```
attribute ram_init_file : string;
attribute ram_init_file of mem : signal is "mem_content.mif";
```

El archivo `Secuenciador/memory.vhd` contiene la memoria de microprograma. Recibe una dirección de 12 bits y entrega una palabra de 98 bits mediante condiciones `if/elsif`. Cambiar el MIF modifica el programa; cambiar esta memoria modifica las operaciones internas de las instrucciones.

3.3. Preparación de la microprogramación con la hoja del profesor

Para preparar las palabras de control se utilizó como referencia el archivo `salidas_68HC11_CISC-practica7.xlsx`, proporcionado por el profesor [1]. La hoja `salidas` reúne las microdirecciones y las señales de cada palabra. Las columnas separan los campos del secuenciador, los acumuladores, la UPA, los registros, la memoria, las interrupciones y el CCR. La columna final, `CW`, concatena los valores de cada fila para formar la cadena de 98 bits que se coloca en `memory.vhd`.

La hoja `entradas` relaciona las condiciones con los códigos del campo de prueba; por ejemplo, la bandera Z corresponde a `10010`. Las hojas de instrucciones y de *fetch* relacionan las señales con las transferencias entre registros. También se consultaron los diagramas de direccionamiento inmediato y extendido y la tabla de líneas de control [2, 3].

El procedimiento consistió en identificar la operación de cada paso, seleccionar los buses y registros que participan, establecer las habilitaciones y escrituras, definir la condición de salida y revisar la cadena binaria resultante. Finalmente se asociaron esas cadenas con las microdirecciones de cada opcode. Las palabras de las rutinas compartidas de la hoja `salidas` coinciden con las utilizadas en el VHDL entregado.

Tabla 1: Campos principales de la palabra de microprograma.

Campo	Bits	Longitud	Función
Prueba	97 : 93	5	Selección de la condición que se evalúa.
VF	92	1	Valor de referencia para la condición.
Instrucción	91 : 90	2	Selección del modo de secuenciamiento.
Liga	89 : 78	12	Dirección de destino de microprograma.
Control	77 : 0	78	Acumuladores, UPA, registros, memoria, interrupciones y banderas.

El espacio lógico direccionable de la memoria de control es de $2^{12} = 4096$ palabras de 98 bits, equivalentes a 401 408 bits lógicos. El VHDL describe únicamente las direcciones necesarias y

una salida por defecto, por lo que esta cifra no representa por sí sola el consumo físico de memoria en la FPGA.

3.4. Búsqueda y decodificación de instrucciones

El ciclo de búsqueda utiliza las microdirecciones `008`, `009` y `00A`. Primero se transfiere el PC al registro de direcciones; después se lee el opcode, se carga en el registro de instrucciones y se incrementa el PC. Finalmente se selecciona la entrada del microprograma correspondiente.

El registro de instrucciones concatena el opcode con cuatro bits en cero:

$$\text{entrada de microprograma} = \text{opcode} \,\|\, \texttt{0}.$$

Por ejemplo, `86` conduce a `860` y `1C` a `1C0`. Los pasos siguientes ocupan direcciones consecutivas hasta regresar a la búsqueda. El retorno puede preparar de nuevo el registro de direcciones y continuar en `009`, sin repetir siempre `008`.

Tabla 2: Instrucciones utilizadas en esta implementación.

Instrucción	Opcode	Bytes	Microdirecciones	Operación principal
LDAB inmediato	`D6`	2	`D60-D63`	$B \leftarrow$ dato inmediato.
LDAA inmediato	`86`	2	`860-863`	$A \leftarrow$ dato inmediato.
ABA inherente	`1B`	1	`1B0-1B2`	$A \leftarrow A + B$.
JMP extendido	`7E`	3	`7E0-7E5`	PC $\leftarrow$ dirección de 16 bits.
CMP inherente	`1C`	1	`1C0-1C2`	Evaluación de $A - B$ y banderas.
JE extendido	`7D`	3	`7D0-7D5`	Salto cuando $Z = 1$.
STAA extendido	`A7`	3	`A70-A77`	$M[\text{dirección}] \leftarrow A$.
LDAA extendido	`96`	3	`960-967`	$A \leftarrow M[\text{dirección}]$.
LDAB extendido	`F6`	3	`F60-F67`	$B \leftarrow M[\text{dirección}]$.

Los opcodes de la Tabla 2 corresponden al procesador del proyecto. La rutina `D60-D63` reproduce la carga inmediata de B que ya existe en `C60-C63`. La práctica del semestre anterior se consultó como referencia de organización [4]; los programas de este reporte emplean las asignaciones verificadas en el VHDL actual.

3.5. Operaciones internas

En las cargas inmediatas se toma el byte que sigue al opcode, se guarda en el acumulador y se incrementa el PC. Para `ABA`, la UPA recibe A y B, calcula su suma y escribe el resultado en A. El acumulador B conserva su dato.

La instrucción `CMP` evalúa $A - B$ sin escribir el resultado en los acumuladores. La bandera Z detecta si la resta fue cero. En `JE`, la microinstrucción `7D3` selecciona la prueba `10010`, correspondiente a Z. Si $Z = 1$, se ejecuta el paso que carga la dirección de destino en el PC; si $Z = 0$, se omite esa carga y el programa continúa después de los dos bytes de dirección.

En el modo extendido se leen primero el byte alto y el byte bajo de la dirección. Después se utiliza esa dirección para leer la RAM en `LDAA/LDAB`, escribirla en `STAA` o cargar el PC en `JMP`. Por ejemplo, `00 18` representa la dirección `0018`, equivalente a 24 en decimal.

4. Programas de prueba

Todos los MIF utilizan `WIDTH=8`, `DEPTH=256`, `ADDRESS_RADIX=UNS` y `DATA_RADIX=HEX`. Las direcciones a la izquierda de los dos puntos son decimales y los bytes a la derecha, hexadecimales. Cada prueba se coloca dentro de un único bloque `CONTENT BEGIN ... END;`.

4.1. Ejercicio 1: suma repetitiva y carga de FF

Se carga $B = 02$ y $A = 00$. Después se ejecuta `ABA` y se regresa a la dirección `0004`, donde está la suma. El algoritmo esperado es:

```
B = 0x02;
A = 0x00;
while (1) {
    A = A + B;
}
```

Código 1: Ejercicio 1: el byte de la posición 7 selecciona la variante.

```
WIDTH=8;
DEPTH=256;
ADDRESS_RADIX=UNS;
DATA_RADIX=HEX;
CONTENT BEGIN
    0 : D6; -- LDAB inmediato
    1 : 02;
    2 : 86; -- LDAA inmediato
    3 : 00;
    4 : 1B; -- A <- A + B
    5 : 7E; -- JMP extendido
    6 : 00;
    7 : 04; -- 04: repetir suma; 08: ir a la carga de FF
    8 : 86; -- LDAA inmediato
    9 : FF;
    [10..255] : 00;
END;
```

Para la segunda prueba se cambió únicamente el destino del salto: la posición 7 contiene `08`. Se realiza una suma y después se ejecuta `LDAA` con el operando `FF`. La copia del MIF incluida en `code` conserva esta segunda variante. El opcode `86` tiene direccionamiento inmediato en el VHDL utilizado, aunque el comentario de la copia original lo identifica como directo.

4.2. Ejercicio 2: comparación de acumuladores

Se utilizaron `CMP` y `JE` para probar $B = 05$ con $A = 05$ y con $A = 09$. El salto condicional conduce a `000A`, donde se carga `63`. Si los datos son diferentes, primero se carga `48` y después `63`:

```
B = 0x05;
A = valor_de_prueba;
if (A != B) {
    A = 0x48;
}
A = 0x63;
```

Código 2: Ejercicio 2 para valores diferentes. Para igualdad, la posición 3 contiene 05.

```
WIDTH=8;
DEPTH=256;
ADDRESS_RADIX=UNS;
DATA_RADIX=HEX;
CONTENT BEGIN
    0 : D6; -- B <- 05
    1 : 05;
    2 : 86; -- A <- 09 (usar 05 para igualdad)
    3 : 09;
    4 : 1C; -- CMP: evaluar A - B
    5 : 7D; -- JE a 000A
    6 : 00;
    7 : 0A;
    8 : 86; -- Carga intermedia si A != B
    9 : 48;
    10 : 86; -- Carga comun a ambas rutas
    11 : 63;
    [12..255] : 00;
END;
```

Este programa no corresponde a un `if/else` con resultados finales distintos. Las dos rutas terminan en `63`; la diferencia se reconoce por el paso intermedio por `48`, por Z después de `CMP` y por el recorrido del PC.

4.3. Ejercicio 3: comparación y almacenamiento en RAM

Se emplearon `LDAB`, `LDAA` y `STAA` extendidos para trabajar con `temp` y `res`, ubicadas en `0017` y `0018`. Estas direcciones corresponden a las posiciones decimales 23 y 24 del MIF.

En la prueba de igualdad ambas variables tienen `0A`; en la de diferencia, `temp`=`0A` y `res`=`1B`. Si son diferentes se escribe `48` en `res`. Posteriormente, ambas rutas escriben `63` y ejecutan un salto a `0014`, que mantiene al programa en un bucle final.

```
B = memoria[0x0017]; // temp
A = memoria[0x0018]; // res
if (A != B) {
    A = 0x48;
    memoria[0x0018] = A;
}
A = 0x63;
memoria[0x0018] = A;
while (1) { }          // JMP a 0x0014
```

Código 3: Ejercicio 3 para valores diferentes. Para igualdad, la posición 24 contiene 0A.

```
WIDTH=8;
DEPTH=256;
ADDRESS_RADIX=UNS;
DATA_RADIX=HEX;
CONTENT BEGIN
    0 : F6; -- LDAB: B <- M[0017]
    1 : 00;
    2 : 17;
    3 : 96; -- LDAA: A <- M[0018]
    4 : 00;
    5 : 18;
    6 : 1C; -- CMP
    7 : 7D; -- JE a 000F
    8 : 00;
    9 : 0F;
    10 : 86; -- A <- 48 si los datos son diferentes
    11 : 48;
    12 : A7; -- M[0018] <- A
    13 : 00;
    14 : 18;
    15 : 86; -- A <- 63 en ambas rutas
    16 : 63;
    17 : A7; -- M[0018] <- A
    18 : 00;
    19 : 18;
    20 : 7E; -- JMP a 0014 (bucle final)
    21 : 00;
    22 : 14;
    23 : 0A; -- temp
    24 : 1B; -- res: usar 0A para igualdad
    [25..255] : 00;
END;
```

5. Simulación y análisis de resultados

Las pruebas se realizaron con el editor de formas de onda de Quartus y ModelSim. Se utilizó un reloj de 10 ns de periodo y un intervalo de observación de 1000 ns. En las capturas, `RESET`, `IRQn` y `XIRQn` permanecen en uno; el reinicio es activo en cero y no se aplicaron solicitudes externas de interrupción. Las señales `ACCA_D`, `ACCB_D`, `PC_D` y `Edo_Pres` permiten seguir los resultados y las secuencias internas.

Para cada prueba se guardó el programa, se recompiló el proyecto y se generó una nueva simulación funcional. Durante la preparación se detectó que los cambios del MIF se guardaban en una copia de la carpeta del reporte, mientras Quartus compilaba la copia del proyecto. Al actualizar el archivo utilizado en la compilación, las formas de onda reflejaron el ejercicio seleccionado.

Las figuras siguientes se presentan en páginas horizontales para conservar la legibilidad de los buses. Se interpretan las transiciones de los acumuladores, las direcciones y la bandera Z, además del valor final.

5.1. Ejercicio 1: suma repetitiva

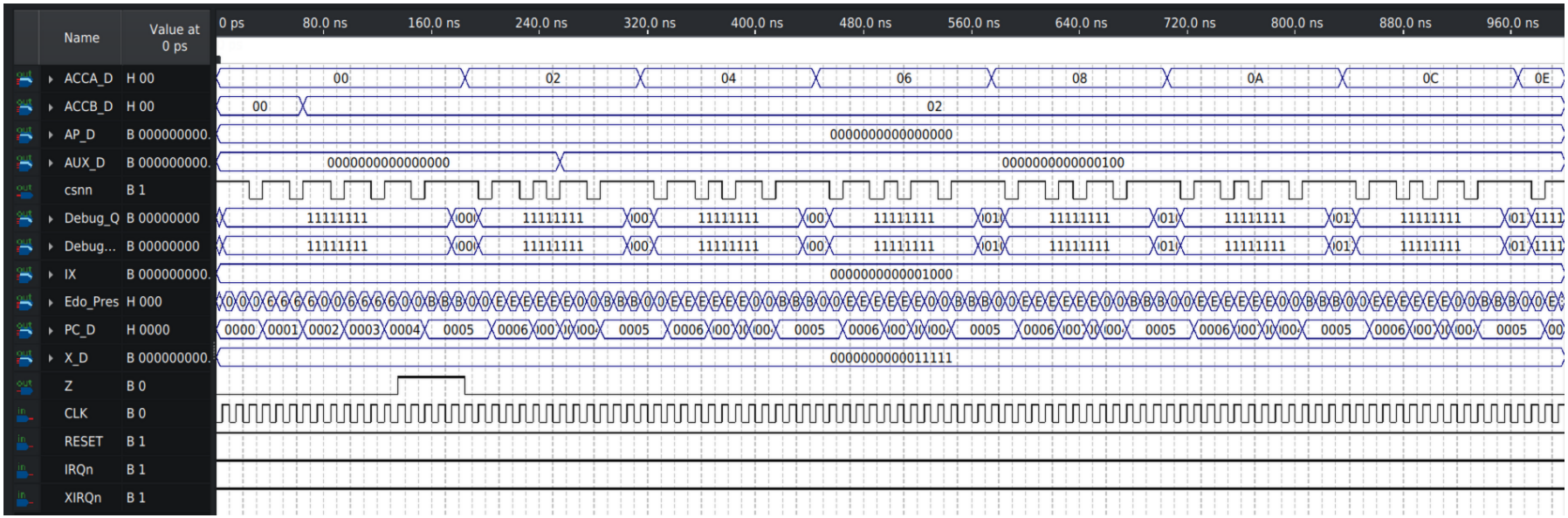


Figura 2: Suma repetitiva con $B = 02$ y salto a 0004.

En la Figura 2, B se carga con 02 y conserva ese valor. A sigue la secuencia 00, 02, 04, 06, 08, 0A, 0C y 0E. El PC vuelve a 0004 después de leer el salto y sus operandos. Esto permite ejecutar ABA repetidamente sin volver a cargar $A = 0$.

Los intervalos entre cambios de A incluyen la búsqueda, la suma y el salto. Por ello, el resultado no cambia en cada periodo del reloj. La captura verifica el incremento de dos en dos durante la ventana observada; no se alcanzó el desbordamiento del acumulador de 8 bits.

5.2. Ejercicio 1: salto a la carga de FF

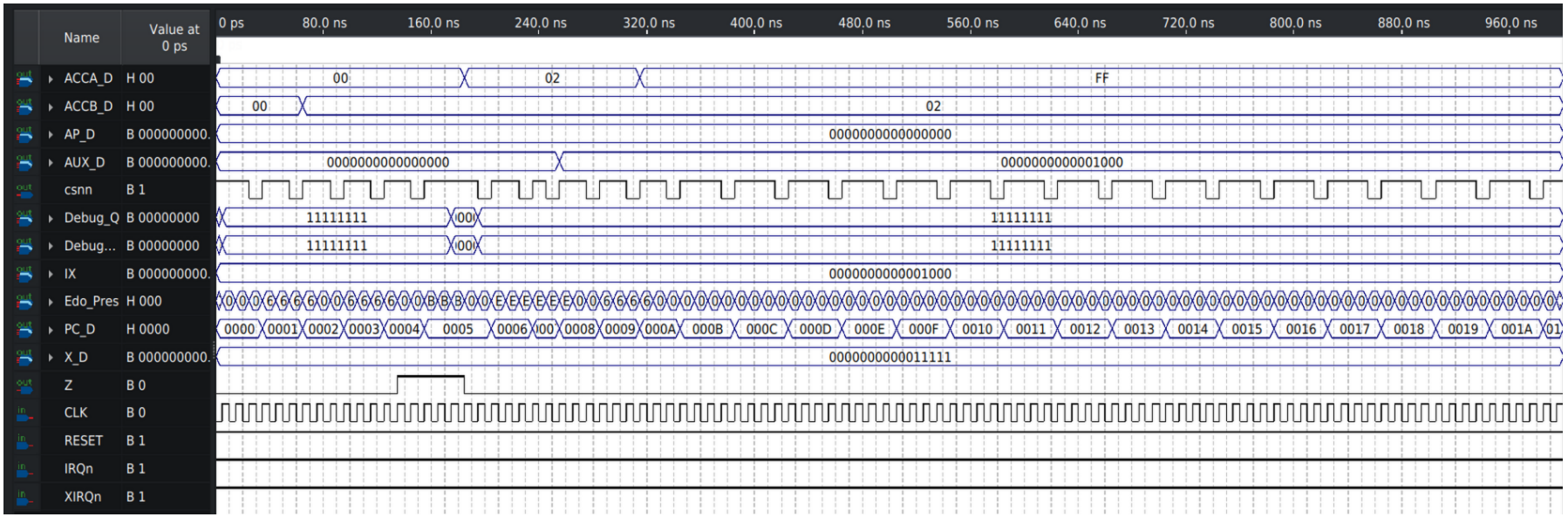


Figura 3: Variante con destino `0008`: después de una suma se carga `FF`.

En la Figura 3, A pasa de `00` a `02` y después a `FF`; B permanece en `02`. El destino `0008` impide regresar a la suma y permite ejecutar la carga inmediata ubicada en las posiciones 8 y 9.

Después de esta carga, el PC continúa por la región inicializada con ceros. La permanencia de A en `FF` no corresponde a una instrucción de alto: no se programó un bucle final en esta variante. El resultado permite comprobar el destino del salto y la carga del operando.

5.3. Ejercicio 2: acumuladores iguales

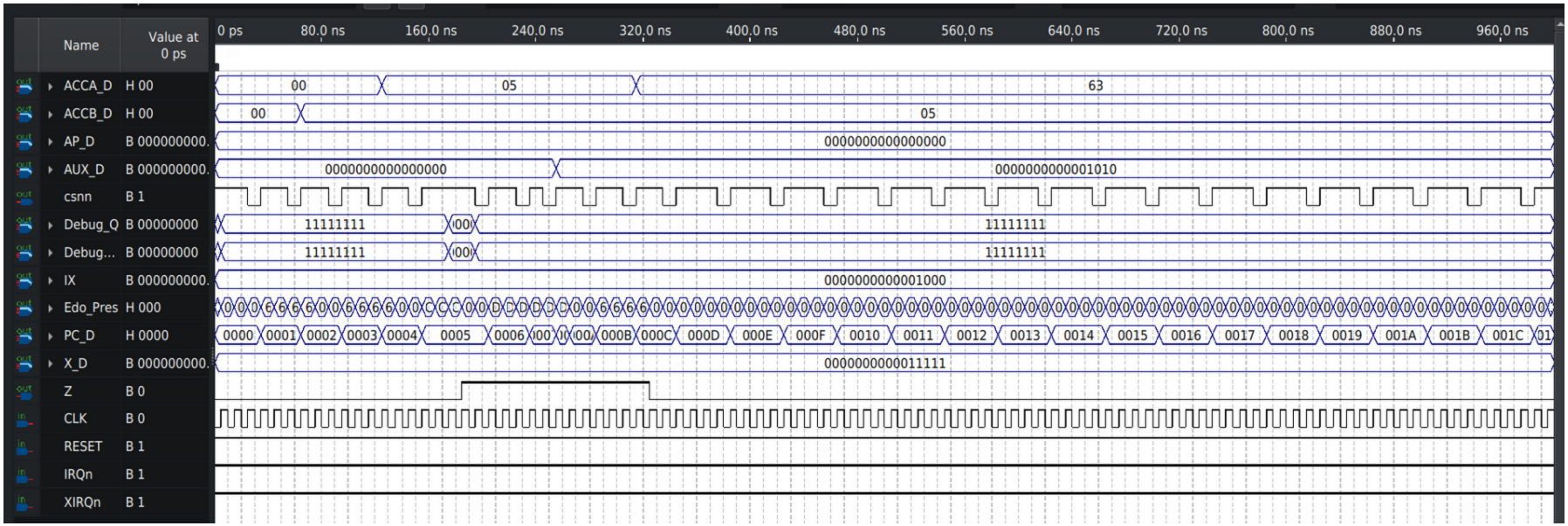


Figura 4: Comparación de $A =$ `05` y $B =$ `05`: se omite la carga de `48`.

En la Figura 4, ambos acumuladores reciben `05`. Después de la comparación, Z toma el valor uno y `JE` conduce al programa a `000A`. A cambia directamente de `05` a `63`; no aparece `48`. Se comprueba que el salto se toma cuando los operandos son iguales.

La bandera Z vuelve a cambiar después de cargar `63`, porque esa instrucción también actualiza las banderas. La condición de igualdad debe revisarse entre `CMP` y `JE`, en lugar de usar únicamente el valor final de Z.

5.4. Ejercicio 2: acumuladores diferentes

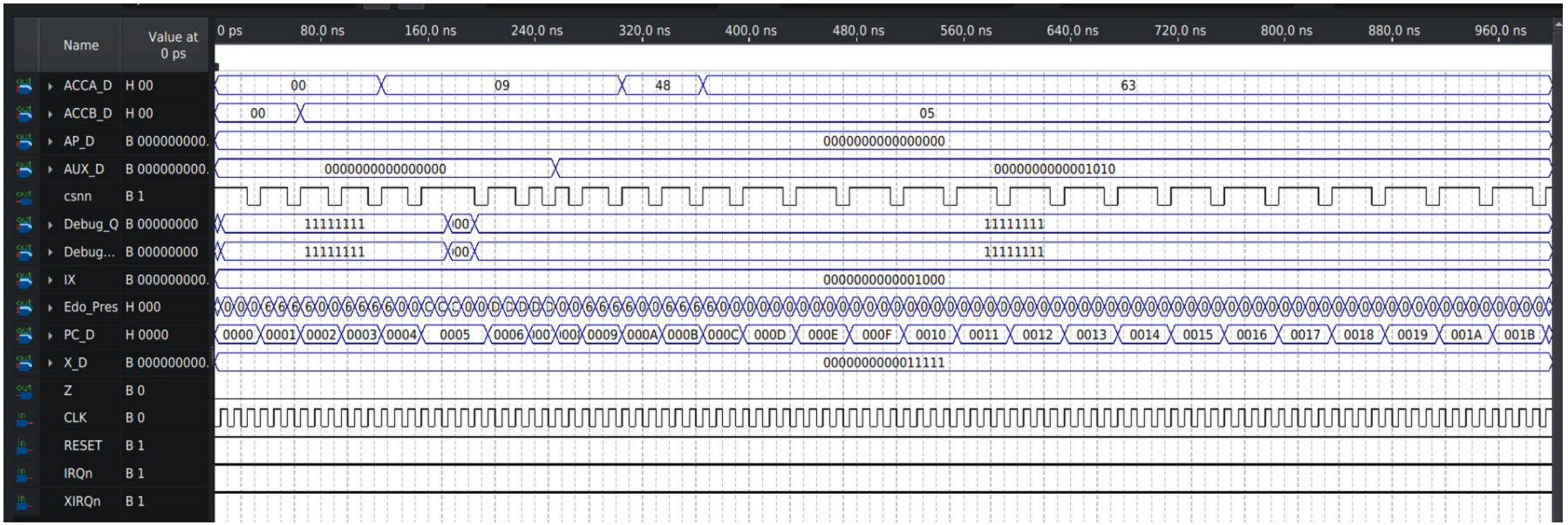


Figura 5: Comparación de $A =$ `09` y $B =$ `05`: A pasa por `48` y termina en `63`.

En la Figura 5, B conserva `05` y A se carga con `09`. La comparación deja $Z = 0$, de modo que no se toma el salto a `000A`. Se ejecuta primero la carga de `48` y luego la de `63`, en concordancia con el Listado 2.

La diferencia respecto a la prueba anterior es el valor intermedio `48` y el recorrido por las posiciones 8 y 9. El valor final `63` es común a ambas pruebas; observarlo de manera aislada no permite decidir si el salto condicional funcionó.

5.5. Ejercicio 3: variables iguales

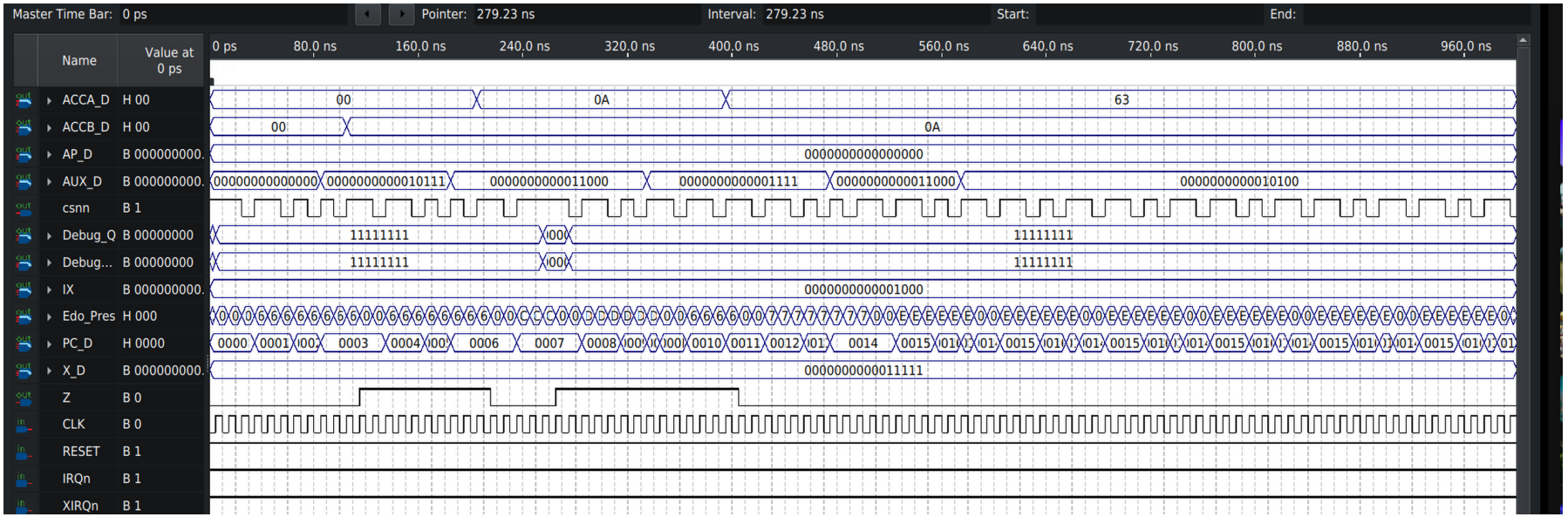


Figura 6: Lectura de `0A` desde ambas variables y salto a la carga común de `63`.

En la Figura 6, ambos acumuladores se cargan con `0A` mediante acceso extendido. La comparación activa Z y se omite la ruta que carga y almacena `48`. A cambia de `0A` a `63`; B mantiene `0A`. Al final, el PC recorre repetidamente el `JMP` situado en `0014`.

El recorrido es consistente con la ejecución de `STAA` para escribir `63` en `0018`. La captura no muestra directamente el contenido interno de la RAM: el dato escrito se deduce del acumulador y de la secuencia de instrucciones, sin una lectura posterior de verificación.

5.6. Ejercicio 3: variables diferentes

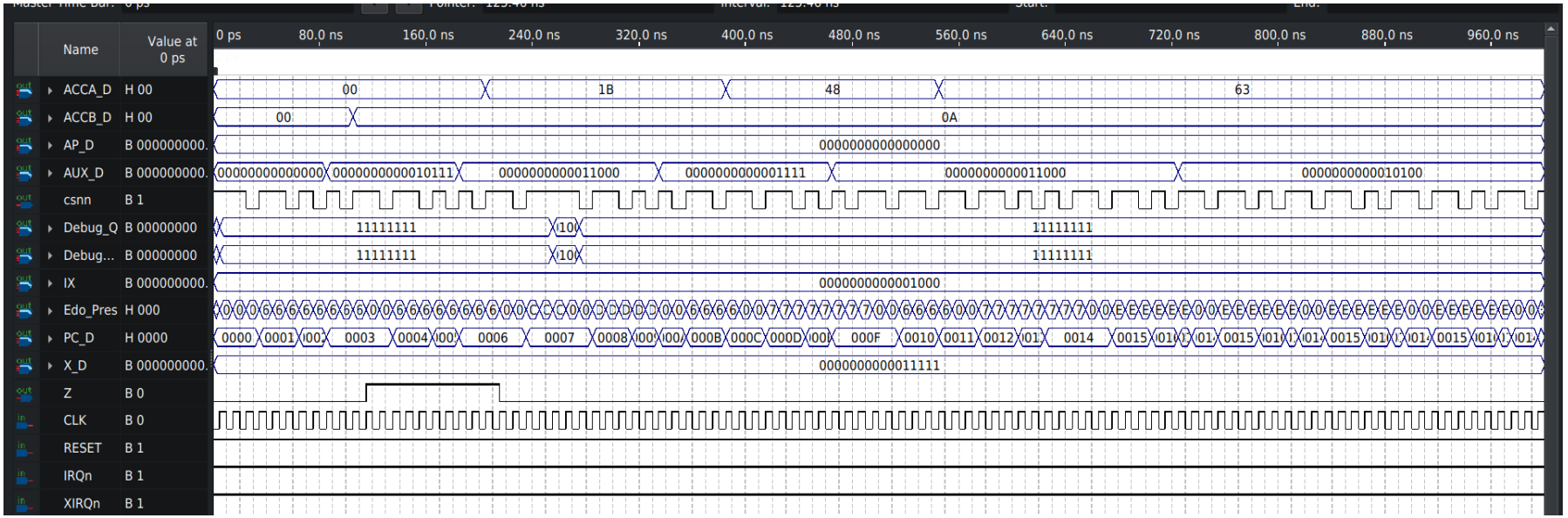


Figura 7: Lectura de $B =$ `0A` y $A =$ `1B`, seguida por las cargas de `48` y `63`.

En la Figura 7, `LDAB` obtiene `0A` de `0017` y `LDAA` obtiene `1B` de `0018`. No se cumple la igualdad. Se recorre la ruta que carga `48` y ejecuta `STAA`; después se carga `63` y se ejecuta el segundo almacenamiento. La secuencia visible en A es `00`, `1B`, `48`, `63`.

El bucle final mantiene el PC alrededor de `0014-0016`, correspondientes al opcode `JMP` y sus operandos. La evidencia permite seguir la ruta de escritura y el dato en A; la captura no incluye una observación directa de la celda `0018`.

6. Discusión de resultados

Tabla 3: Resultados observados en las seis pruebas.

Prueba	Secuencia en A	Comprobación principal
1: suma	`00,02,04,06,08,0A,0C,0E`	Repetición de `ABA` con $B =$ `02`.
1: carga de FF	`00,02,FF`	Salto a `0008` y carga inmediata.
2: iguales	`00,05,63`	`JE` omite la carga de `48`.
2: diferentes	`00,09,48,63`	Continuación por la ruta intermedia.
3: iguales	`00,0A,63`	Lecturas extendidas y omisión de la primera escritura.
3: diferentes	`00,1B,48,63`	Lecturas extendidas y recorrido por ambas escrituras.

Las formas de onda muestran la coordinación entre el PC y las microdirecciones. La suma requiere varias transiciones internas antes de actualizar A. Una instrucción extendida debe obtener dos bytes de dirección antes de realizar el acceso o el salto. Esta diferencia explica por qué una instrucción del programa ocupa varios ciclos del secuenciador.

Las pruebas de comparación muestran que una bandera describe el resultado de la operación que la actualizó más recientemente. Después de `CMP`, Z sirve para decidir el salto; después de una carga puede cambiar. También se observó que dos ejecuciones pueden terminar con el mismo acumulador y haber seguido rutas distintas, como ocurre con `63` en los ejercicios 2 y 3.

El alcance de la comprobación es la simulación funcional. Las imágenes no documentan una implementación física ni una lectura de retorno de la RAM. La referencia anterior incluye un cuarto ejercicio; éste no se presenta como realizado porque los códigos y las capturas disponibles corresponden a los tres ejercicios descritos.

7. Conclusiones

Percival Ulises Espinoza Matamoros. La práctica permitió relacionar las instrucciones del programa con las operaciones internas del procesador. La hoja de cálculo del profesor fue útil para organizar las señales y revisar las palabras de microprograma antes de colocarlas en VHDL. Al seguir las microdirecciones y los acumuladores, se pudo comprobar cómo una carga, una suma o un salto requieren varios pasos coordinados.

Oscar Iván Montiel Juárez. En esta práctica comprendimos la diferencia entre cambiar el programa de la RAM y modificar la memoria de control. También fue necesario comprobar qué archivo utilizaba Quartus, ya que editar otra copia del MIF mantenía los mismos resultados aunque se volviera a compilar. Una vez corregida la ruta, las pruebas mostraron la suma repetitiva, la carga de `FF` y las dos rutas de comparación según Z.

Amy Valentina Veraza García. Las pruebas con datos iguales y diferentes ayudaron a interpretar `CMP` y `JE`. El valor final de A no fue suficiente para distinguir las rutas, porque ambas terminaban en `63`; fue necesario observar el paso por `48`, la bandera y el PC. El direccionamiento extendido permitió relacionar las variables con sus direcciones en RAM y con las instrucciones de carga y almacenamiento.

Referencias

[1] Cruz Estrada, J. C. *salidas_68HC11_CISC-practica7.xlsx*. Material de apoyo del Laboratorio de Organización y Arquitectura de Computadoras. Hojas `salidas`, `entradas`, `LDAB(INM)`, `LDAA(INM)` y `fetch`.

[2] Laboratorio de Organización y Arquitectura de Computadoras. *Diagramas de direccionamiento inmediato y extendido*. Archivos `inmediato.pdf` y `extendido.pdf`, material proporcionado por el profesor.

[3] Laboratorio de Organización y Arquitectura de Computadoras. *Líneas de control y arquitectura del procesador 68HC11*. Archivos `lineas_de_control_update.pdf` y `68HC11.pdf`, material proporcionado por el profesor.

[4] Jiménez Treviño, E. C., y Martinez Perez, B. E. *Práctica 7. Procesador CISC 68HC11*. Reporte del semestre 2026-1, 18 de noviembre de 2025. Referencia de organización y ejercicios.

A. Código de la memoria de control

El listado incorpora directamente `code/memory.vhd`. Se omite únicamente el encabezado de comentarios de formato y ejemplos que precede a los estados activos; se conservan la declaración de la entidad y todas las rutinas implementadas. La numeración corresponde al listado mostrado.

Código 4: Memoria de microprograma de la práctica.

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.STD_LOGIC_ARITH.ALL;
4  use IEEE.STD_LOGIC_UNSIGNED.ALL;
5
6  entity memory is
7  Port (
8    dir : in STD_LOGIC_VECTOR (11 downto 0);
9    data : out STD_LOGIC_VECTOR (97 downto 0));
10 end memory;
11
12 architecture Behavioral of memory is
13
14 begin
15   process(dir)
16   begin
17     --INITIALIZATION STATES (000 - 007) HARD CODE
18     if(dir=    X"000") then data <= "
       1100000100000000100010010010000000000011011100001110001110001110001111111000000000000000000000000010";
19
20     --FETCH
21     elsif(dir= X"008") then data <= "
       0000000000000000000010010010000000000011011100001110001110000110000111111000000000000000000000000010";
22     elsif(dir= X"009") then data <= "
       0000000000000000000000010010000000000011011100001110001110001110011111101000000000000000000000000010";
23     elsif(dir= X"00A") then data <= "
       0000001000000000000010010010000000000011011100001110001110001110001111111000000000000000000000000010";
24
25     --LDAA (INM)
26     elsif(dir= X"860") then data <= "
       0000000000000000000010010010000000000011011100001110001110000110000111111000000000000000000000000010";
27     elsif(dir= X"861") then data <= "
       0000000000000000000010010100000000000011011100001110001110001110011111101000000000000000000000000010";
28     elsif(dir= X"862") then data <= "
       0111111100000000000010010010000000000011011100001110001110001110001111111000000001001100011000010";
29     elsif(dir= X"863") then data <= "
       1100000100000000100110010010000000000011011100001110001110000110000111111000000000000000000000000010";
30
31     --LDAB (INM)
32     elsif(dir= X"C60") then data <= "
       0000000000000000000010010010000000000011011100001110001110000110000111111000000000000000000000000010";
33     elsif(dir= X"C61") then data <= "
       0000000000000000000010100010000000000011011100001110001110001110011111101000000000000000000000000010";
34     elsif(dir= X"C62") then data <= "
       0111111100000000000010010010000000000011011100001110001110001110001111111000000010010100011000010";
35     elsif(dir= X"C63") then data <= "
       1100000100000000100110010010000000000011011100001110001110000110000111111000000000000000000000000010";
36
37     --ABA (INH)
38     elsif(dir= X"1B0") then data <= "
       0000000000000000000011111110000000000011111110000111000111000111000111111100000000000000000000000010";
39     elsif(dir= X"1B1") then data <= "
       0111111100000000000010010100000000000011100001110001110001110001111111000000000000000001111010010";
40     elsif(dir= X"1B2") then data <= "
       1100000100000000100110010010000000000011011100001110001110000110000111111000000000000000000000000010";
41
42     --JMP (EXT)
43     elsif(dir= X"7E0") then data <= "
       0000000000000000000010010010000000000011011100001110001110000110000111111000000000000000000000000010";
44     elsif(dir= X"7E1") then data <= "
       0000000000000000000010010010000000000011011100001011001110001110011111101100000000000000000000000010";
45     elsif(dir= X"7E2") then data <= "
```

```
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
46    elsif(dir= X"7E3") then data <= "
      00000000000000000000100100100000000000110111000011001111100011100111111010000000000000000000000010";
47    elsif(dir= X"7E4") then data <= "
      01111111000000000000100100100000000000110111000010000011100010010111111110000000000000000000000010";
48    elsif(dir= X"7E5") then data <= "
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
49
50    --LDAB (INM)
51    elsif(dir= X"D60") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
52    elsif(dir= X"D61") then data <= "
      00000000000000000000101000100000000000110111000011100011100011100111111010000000000000000000000010";
53    elsif(dir= X"D62") then data <= "
      01111111000000000000100100100000000000110111000011100011100011100111111100000001001010000111000010";
54    elsif(dir= X"D63") then data <= "
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
55
56    --CMP (INH)
57    elsif(dir= X"1C0") then data <= "
      00000000000000000000111111100000100001111111000011100011100011100011111110000000000000000000000011";
58    elsif(dir= X"1C1") then data <= "
      01111111000000000000100100100000000000000111000011100011100011100011111110000000000000000001111000010";
59    elsif(dir= X"1C2") then data <= "
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
60
61    --JE (EXT)
62    elsif(dir= X"7D0") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
63    elsif(dir= X"7D1") then data <= "
      00000000000000000000100100100000000000110111000010110011100011100111111011000000000000000000000010";
64    elsif(dir= X"7D2") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
65    elsif(dir= X"7D3") then data <= "
      10010001011111010101100100100000000000110111000011001111100011100111111010000000000000000000000010";
66    elsif(dir= X"7D4") then data <= "
      01111111000000000000100100100000000000110111000010000011100010010111111110000000000000000000000010";
67    elsif(dir= X"7D5") then data <= "
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
68
69    --STA (EXT)
70    elsif(dir= X"A70") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
71    elsif(dir= X"A71") then data <= "
      00000000000000000000100100100000000000110111000010110011100011100111111011000000000000000000000010";
72    elsif(dir= X"A72") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
73    elsif(dir= X"A73") then data <= "
      00000000000000000000100100100000000000110111000011001111100011100111111010000000000000000000000010";
74    elsif(dir= X"A74") then data <= "
      00000000000000000000100100100000000000110111000011000111000111000011111100000000000000000000000010";
75    elsif(dir= X"A75") then data <= "
      00000000000000000000100101100000000000110111000011100011100011100011111100000000000000000000000010";
76    elsif(dir= X"A76") then data <= "
      01111111000000000000100100100000000000110111000011100011100011100011111100000001001100001110000010";
77    elsif(dir= X"A77") then data <= "
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
78
79    -- LDAA (EXT)
80    elsif(dir= X"960") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
81    elsif(dir= X"961") then data <= "
      00000000000000000000100100100000000000110111000010110011100011100111111011000000000000000000000010";
82    elsif(dir= X"962") then data <= "
      00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
83    elsif(dir= X"963") then data <= "
      00000000000000000000100100100000000000110111000011001111100011100111111010000000000000000000000010";
84    elsif(dir= X"964") then data <= "
      00000000000000000000100100100000000000110111000011000111000111000011111100000000000000000000000010";
85    elsif(dir= X"965") then data <= "
      00000000000000000000100101000000000000110111000011100011100011100011111010000000000000000000000010";
86    elsif(dir= X"966") then data <= "
      01111111000000000000100100100000000000110111000011100011100011100011111100000001001100001110000010";
87    elsif(dir= X"967") then data <= "
```

```
      11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
88
89      -- LDAB (EXT)
90      elsif(dir= X"F60") then data <= "
          00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
91      elsif(dir= X"F61") then data <= "
          00000000000000000000100100100000000000110111000010110011100011100111111011000000000000000000000010";
92      elsif(dir= X"F62") then data <= "
          00000000000000000000100100100000000000110111000011100011100001100001111110000000000000000000000010";
93      elsif(dir= X"F63") then data <= "
          00000000000000000000100100100000000000110111000011001111100011100111111010000000000000000000000010";
94      elsif(dir= X"F64") then data <= "
          00000000000000000000100100100000000000110111000001100011100011100001111110000000000000000000000010";
95      elsif(dir= X"F65") then data <= "
          00000000000000000000101000100000000000110111000011100011100011100001111101000000000000000000000010";
96      elsif(dir= X"F66") then data <= "
          01111111000000000000100100100000000000110111000011100011100011100001111111000000001001100011000010";
97      elsif(dir= X"F67") then data <= "
          11000001000000001001100100100000000000110111000011100011100001100001111110000000000000000000000010";
98
99
100     else data <= "00000000000000000000100100100000000000110111000011100011100011100001111111000000000000000000000010";
101
102     end if;
103   end process;
```